
HỌC PHẦN NÂNG CAO VỚI C++

BỘ MÔN CÔNG NGHỆ THÔNG TIN

CHƯƠNG 1: GIỚI THIỆU VỀ LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

BỘ MÔN CÔNG NGHỆ THÔNG TIN

- ❖ 1.1. Các khái niệm về lập trình hướng đối tượng
- ❖ 1.2. Lập trình phi cấu trúc
- ❖ 1.3. Lập trình có cấu trúc
- ❖ 1.4. Lập trình hướng đối tượng
- ❖ 1.5. Một số cơ chế trong lập trình hướng đối tượng
- ❖ 1.6. Trừu tượng hóa
- ❖ 1.7. Sự đóng gói
- ❖ 1.8. Tính kế thừa
- ❖ 1.9. Tính đa hình
- ❖ Bài tập chương 1

Các khái niệm về lập trình hướng đối tượng

❖ Kỹ thuật lập trình

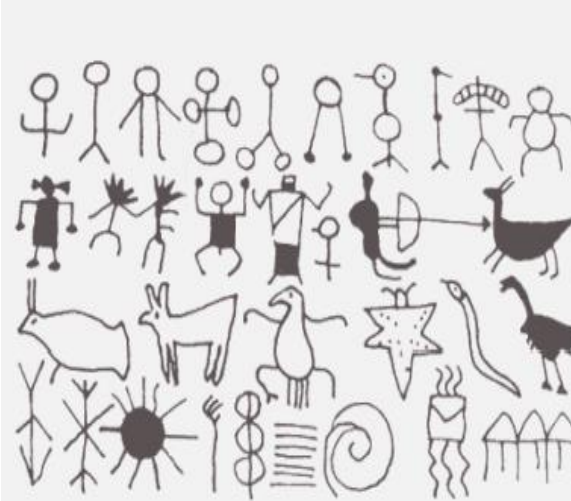
- Lập trình hướng đối tượng là một kỹ thuật lập trình. Vậy kỹ thuật lập trình là gì?
- Kỹ thuật lập trình: Kỹ thuật thực thi một giải pháp phần mềm (cấu trúc dữ liệu + giải thuật) dựa trên nền tảng một phương pháp luận (methodology) và một hoặc nhiều ngôn ngữ lập trình phù hợp với yêu cầu đặc thù của ứng dụng.
- **Phương pháp luận**
 - Các mô thức lập trình
 - Các ý tưởng, thuật toán để giải quyết vấn đề
 - Phong cách trình bày trong lập trình
 - Văn hóa lập trình

❖ Ngôn ngữ lập trình

- Mô thức - nguyên tắc chung cơ bản
- Cú pháp - xác định cái gì là hợp lệ trong mã nguồn
- Ngữ nghĩa - ngữ pháp của ngôn ngữ lập trình

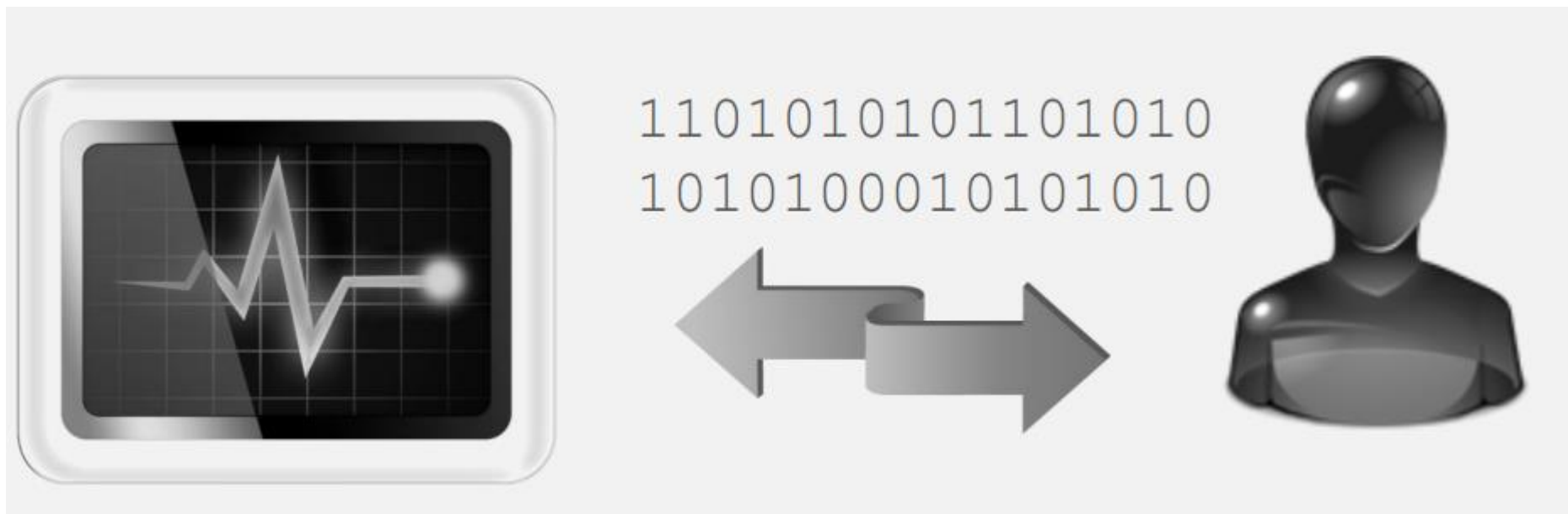
❖ Ngôn ngữ?

- Phương tiện để giao tiếp
- Hệ thống ký hiệu để diễn đạt



Ngôn ngữ lập trình

- ❖ **Ngôn ngữ máy:** Các chỉ thị được thể hiện bằng các chữ số nhị phân 1 và 0.



❖ Ngôn ngữ lập trình

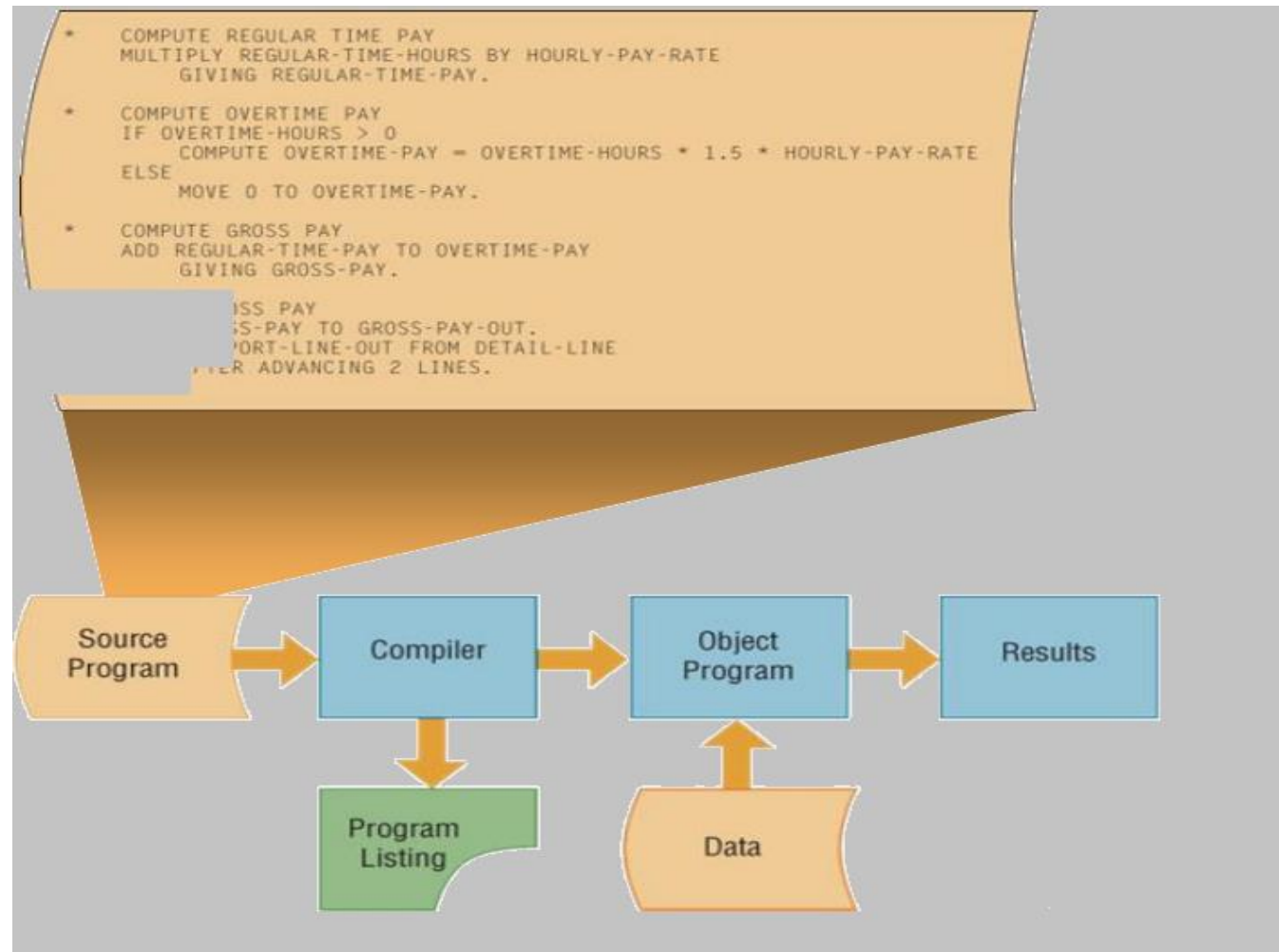
- Là ngôn ngữ được chuẩn hóa
- Cả con người và máy tính có thể đọc và hiểu được
- Sử dụng chương trình dịch tương ứng để giao tiếp với máy tính



```
for (int j = 0; j < loc; j++) res[j] = buf[j];  
return res;  
  
public void checkRes(int[] res) {  
    for (int i = 0; i < res.length; i++) {  
        if (res[i] != checkRes[i]) {  
            res[i] = checkRes[i];  
        }  
    }  
}  
  
public void decodeMessage(int[] res) {  
    for (int i = 0; i < MAX_RES_LEN; i++) buf[i] = 0;  
    int loc = 0;  
    for (int i = 0; i < res.length; i++) {  
        if (res[i] != 0) {  
            buf[loc++] = res[i];  
        }  
    }  
    int len = loc;  
    int[] res2 = new int[MAX_RES_LEN];  
    for (int i = 0; i < len; i++) {  
        res2[i] = buf[i];  
    }  
    extractMessage(res2);  
}  
  
public int[] extractMessage(int[] res) {  
    for (int i = 0; i < MAX_RES_LEN; i++) buf[i] = 0;  
    int loc = 0, i = 0;  
    while (i < res.length) {  
        if (res[i] != 0) {  
            buf[loc++] = res[i];  
        }  
        i++;  
    }  
    int len = loc;  
    int[] res2 = new int[MAX_RES_LEN];  
    for (int i = 0; i < len; i++) {  
        res2[i] = buf[i];  
    }  
    return res2;  
}
```

Trình dịch

- ❖ **Trình dịch – Compiler:** Là chương trình thực hiện biên dịch toàn bộ chương trình nguồn thành mã máy trước khi thực hiện



Sự phát triển của ngôn ngữ lập trình



Sự phát triển của ngôn ngữ lập trình

❖ Lập trình tuyến tính

- Còn gọi là lập trình phi cấu trúc
- Giải quyết các bài toán tương nhỏ, đơn giản

❖ Đặc điểm:

- Chỉ gồm một chương trình chính
- Gồm một dãy tuần tự các câu lệnh
- Chương trình ngắn, ít hơn 100 dòng

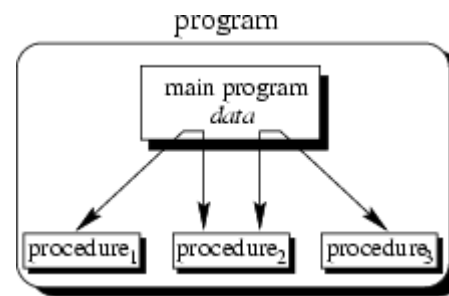


❖ Nhược điểm:

- Không sử dụng lại được các đoạn mã
- Không có khả năng kiểm soát phạm vi truy xuất dữ liệu
- Mọi dữ liệu trong chương trình là toàn cục
- Dữ liệu có thể bị sửa đổi ở bất cứ vị trí nào trong chương trình

❖ Không đáp ứng được việc triển khai phần mềm

- ❖ Ra đời vào những năm 70
- ❖ Chương trình được chia nhỏ thành chương trình con:
 - Thủ tục (Procedure)
 - Hàm (Function)
- ❖ Các chương trình con:
 - Độc lập với nhau và có dữ liệu riêng
 - Trao đổi qua: tham số và biến toàn cục
 - Xây dựng chương trình dựa trên các hàm/thủ tục/chương trình con
 - Dữ liệu và xử lý (hàm) tách rời nhau.
 - Các hàm không bắt buộc phải tuân theo một cách thức chung truy cập vào dữ liệu



❖ Xuất hiện khái niệm trừu tượng hoá

- Là khả năng quan sát sự vật mà:
 - Không quan tâm tới các chi tiết không quan trọng bên trong
 - Không quan tâm tới việc thực hiện như thế nào

❖ Ngôn ngữ lập trình cấu trúc:

- C, Pascal
- Foxpro ...

Lập trình cấu trúc

```
#include <iostream>
using namespace std;

// Hàm tính điểm trung bình
float tinhDiemTrungBinh(float toan, float ly, float hoa) {
    return (toan + ly + hoa) / 3;
}

// Hàm xếp loại học sinh
void xepLoai(float dtb) {
    cout << "Diem trung binh: " << dtb << endl;
    if (dtb >= 8) {
        cout << "Xep loai: Gioi" << endl;
    } else if (dtb >= 6.5) {
        cout << "Xep loai: Kha" << endl;
    } else if (dtb >= 5) {
        cout << "Xep loai: Trung binh" << endl;
    } else {
        cout << "Xep loai: Yeu" << endl;
    }
}

int main() {
    float toan, ly, hoa;

    // Nhập điểm
    cout << "Nhap diem Toan: ";
    cin >> toan;
    cout << "Nhap diem Ly: ";
    cin >> ly;
    cout << "Nhap diem Hoa: ";
    cin >> hoa;

    // Gọi hàm tính điểm trung bình
    float dtb = tinhDiemTrungBinh(toan, ly, hoa);

    // Gọi hàm xếp loại
    xepLoai(dtb);

    return 0;
}
```

Sự phát triển của ngôn ngữ lập trình

❖ Lập trình tuần tự

- Mã nguồn được viết dưới dạng "đầu tiên làm thế này, sau đó làm thế kia"
- Khó sửa lỗi, bảo trì.

```
#include <iostream>
using namespace std;

int main() {
    // Khai báo biến
    float chieu_dai, chieu_rong, dien_tich;

    // Nhập dữ liệu từ người dùng
    cout << "Nhập chiều dài: ";
    cin >> chieu_dai;

    cout << "Nhập chiều rộng: ";
    cin >> chieu_rong;

    // Tính diện tích
    dien_tich = chieu_dai * chieu_rong;

    // In kết quả
    cout << "Diện tích hình chu nhật là: " << dien_tich << endl;

    return 0;
}
```

Nhược điểm của lập trình truyền thống

- ❖ Chương trình khó kiểm soát
- ❖ Khó khăn trong việc bổ sung, nâng cấp chương trình
- ❖ Khi thay đổi, bổ sung dữ liệu dùng chung thì phải thay đổi gần như tất cả thủ tục/hàm liên quan
- ❖ Khả năng sử dụng lại các đoạn mã chưa nhiều
- ❖ Không mô tả đầy đủ, trung thực hệ thống trong thực tế

ĐỐI TƯỢNG

Đối tượng là gì?

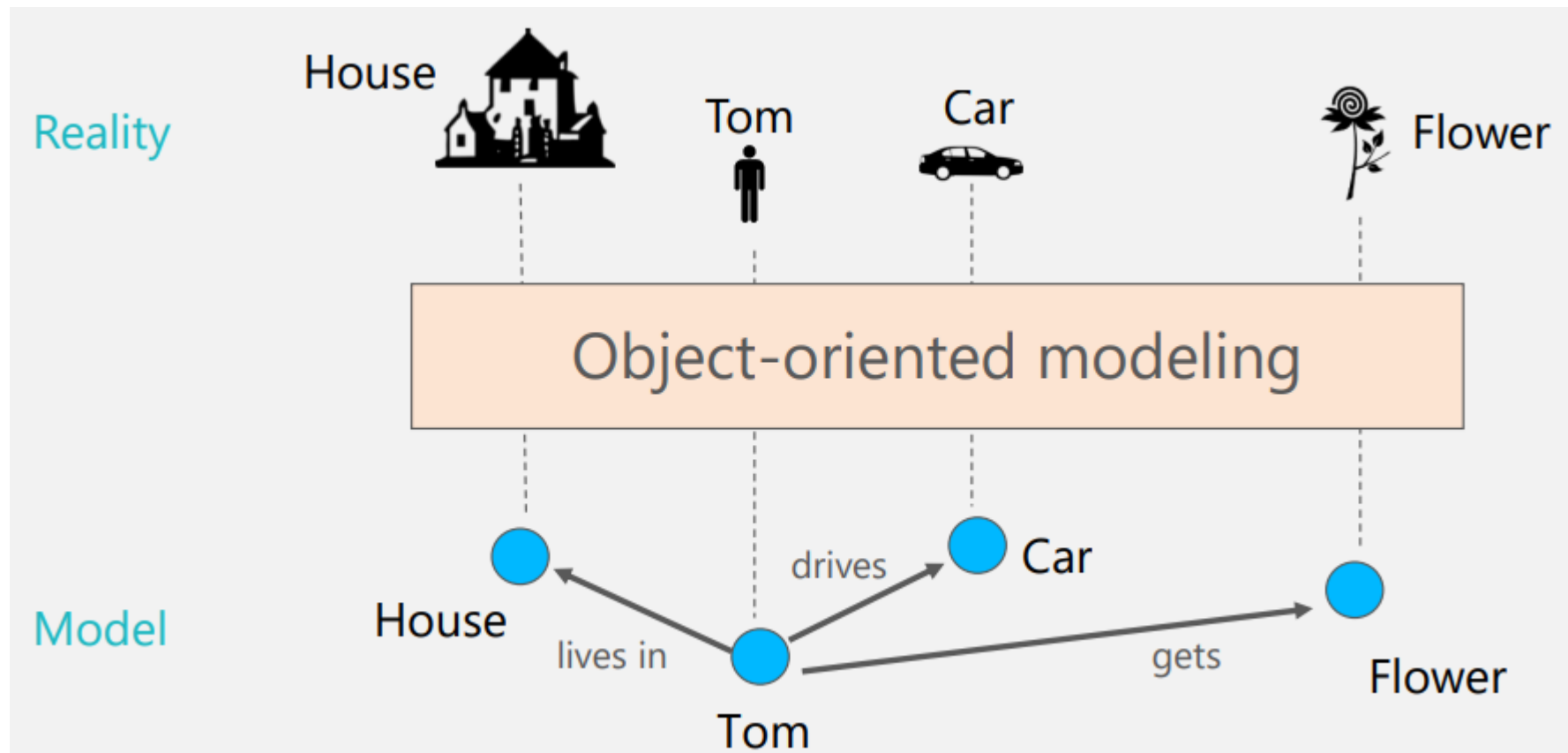
"Tất cả mọi thứ đều là đối tượng"

-Alan Kay



LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

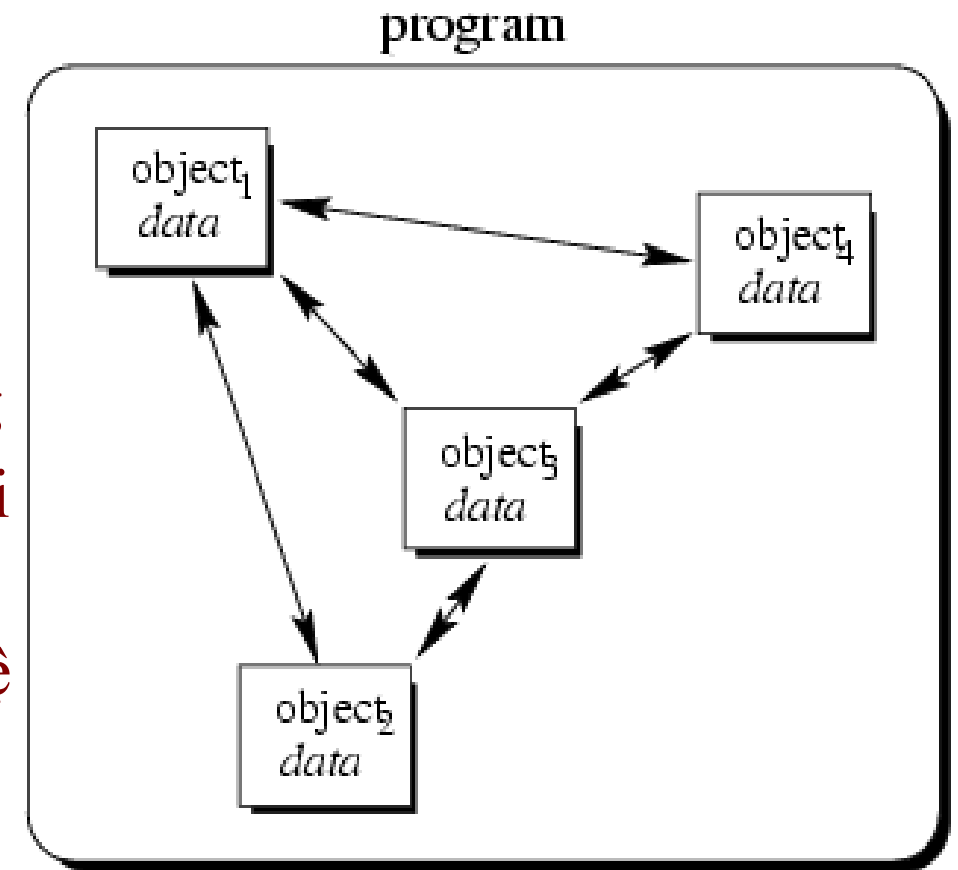
- ❖ Cõi chương trình phần mềm là một tập hợp các đối tượng tương tác với nhau



LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

❖ Là phương pháp lập trình:

- Mô tả chính xác các đối tượng trong thế giới
- Lấy đối tượng làm nền tảng xây dựng thuật toán
- Thiết kế xoay quanh dữ liệu của hệ thống
- Chương trình được chia thành các lớp đối tượng
- Dữ liệu được đóng gói, che dấu và bảo vệ
- Đối tượng làm việc với nhau qua thông báo
- Chương trình được thiết kết theo cách từ dưới lên (bottom-up)



LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

- ❖ Mỗi đối tượng trong chương trình có các dữ liệu độc lập của mình và chiếm bộ nhớ riêng của mình.
- ❖ Mỗi đối tượng đều có dạng đặc trưng của lớp các đối tượng đó.
- ❖ Tất cả các đối tượng thuộc về cùng một lớp đều có các hành vi giống nhau

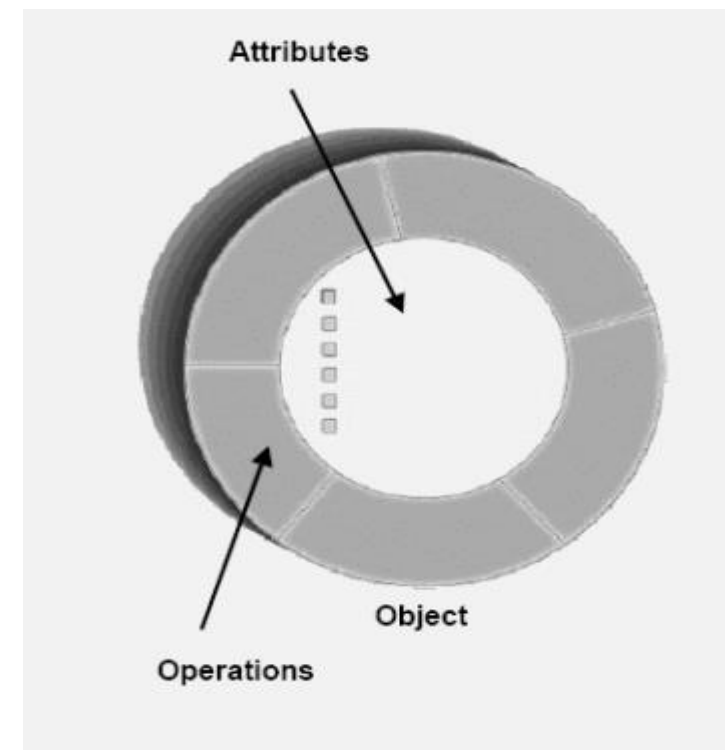
- Alan Kay

- ❖ Trong thế giới thực
 - *VD: Sinh viên, ô tô, xe máy, lớp học, tài khoản ngân hàng, màu sắc, ...*
- ❖ Mỗi đối tượng đều có:
 - Các thông tin, trạng thái
 - Ví dụ: Các thông tin về ô tô có số khung, số máy, hang sản xuất, màu sắc, tốc độ, năm sx, ...
- ❖ Các hoạt động
 - *Ví dụ: Tăng ga, phanh, giảm tốc, đi đến một địa điểm, ...*

ĐỐI TƯỢNG

	Trạng thái	Hành vi	
Con chó	Tên Màu sắc Giống chó Cảm xúc	Sủa Vẫy đuôi Ăn Chạy	
Ô tô	Hãng sản xuất Kích thước Màu sắc Giá tiền	Tăng tốc Giảm tốc Đâm	
Tài khoản	Tên tài khoản Số tài khoản Ngân hàng Số dư	Rút tiền Gửi tiền Kiểm tra số dư	

- ❖ Mô hình hóa vào trong lập trình: Đóng gói thành **trạng thái (state)** và **hành vi (behavior)**.
 - Trạng thái được biểu diễn bởi các **thuộc tính (attributes)** và **các mối quan hệ (relationships)**.
 - Hành vi được biểu diễn bởi **các thao tác (operations)** hay **phương thức (methods)**.



TRẠNG THÁI VÀ HÀNH VI

❖ Trạng thái:

- Trạng thái là dữ liệu của đối tượng.
- Trạng thái của một đối tượng có thể thay đổi theo thời gian.

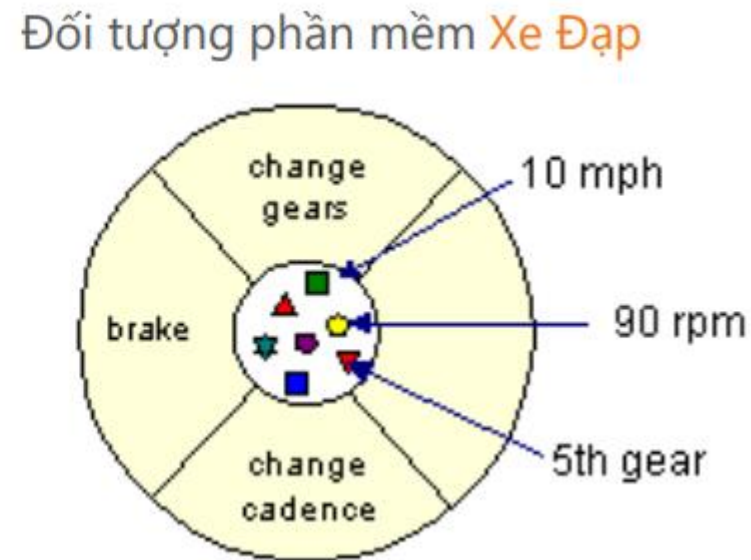
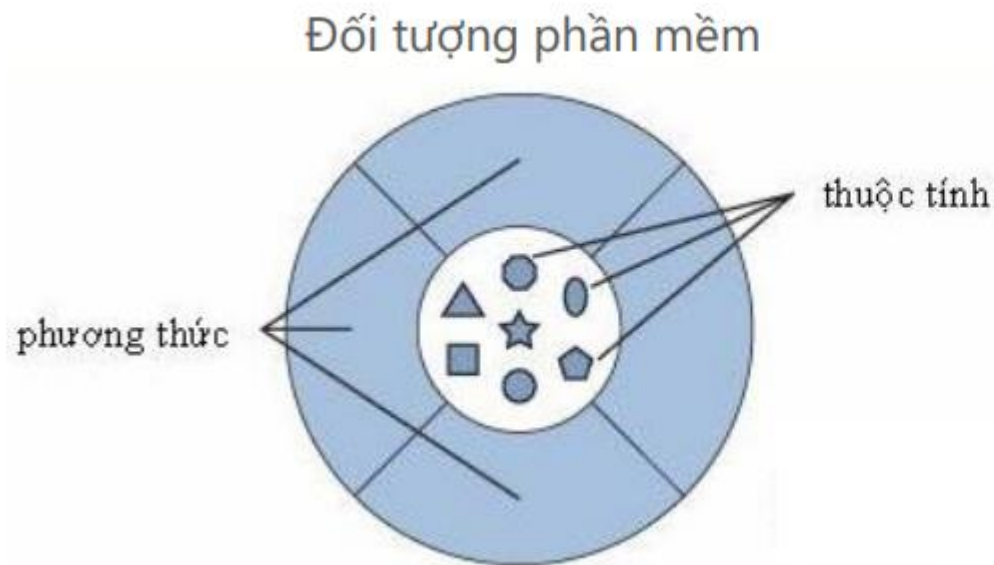
❖ Hành vi: các hành động mà đối tượng thực hiện

Đối tượng “Quạt điện”

Trạng thái	Hành vi
Màu sắc: trắng	Bật quạt
Tốc độ: số 2	Tắt quạt
Trạng thái bật/tắt: đang bật	Tăng tốc độ
Hướng quay: trái-phải	Giảm tốc độ
	Quay trái/phải

ĐỐI TƯỢNG PHẦN MỀM

- ❖ Đối tượng (object) là một thực thể phần mềm bao bọc các thuộc tính và các phương thức liên quan:
 - Thuộc tính được xác định bởi giá trị cụ thể gọi là **thuộc tính thể hiện**
 - Một đối tượng cụ thể được gọi là **một thể hiện**.



THUỘC TÍNH VÀ PHƯƠNG THỨC

❖ Thuộc tính bao gồm:

- Hằng, biến
- Tham số nội tại
- Thuộc tính được xác định kiểu, gồm:
- Kiểu cổ điển
- Kiểu do người dùng định nghĩa

❖ Phương thức là:

- Các hàm nội tại của đối tượng
- Có kiểu trả về
- Tên gọi khác: hàm thành viên

Đối tượng "Xe ô tô"

Thuộc tính (Attributes / State): Mô tả đặc điểm của đối tượng	Phương thức (Methods / Behavior): Mô tả hành động hoặc chức năng mà đối tượng có thể thực hiện.
Màu sắc: đỏ	Khởi động()
Số chỗ ngồi: 4	Tăng tốc()
Tốc độ tối đa: 180 km/h	Phanh()
Nhiên liệu: xăng	Rẽ trái()
Biển số: 30A-88888	Rẽ phải() ...

LỚP VÀ LỚP CON

- ❖ **Lớp:** là tập hợp các đối tượng có cùng thuộc tính và hành vi
 - Là bản thiết kế hoặc bản mẫu mô tả một cấu trúc dữ liệu gồm:
 - Các thành phần dữ liệu
 - Các phương thức
 - Lớp được sử dụng như kiểu dữ liệu người dùng định nghĩa
- ❖ **Lớp con:**
 - Là lớp thông thường
 - Có thêm tính chất kế thừa đặc tính của lớp khác

LỚP VÀ ĐỐI TƯỢNG

Lớp Xe ô tô



❖ **Lớp cha: Nhân viên ngân hàng**

- Thuộc tính chung (trạng thái): Họ tên, Mã nhân viên, Chức vụ, Ngày vào làm
- Hành vi chung: Chăm công, Nhận lương, Tuân thủ quy định nội bộ

❖ **Các lớp con:**

- **Giao dịch viên**
 - **Thuộc tính riêng:** Quầy giao dịch, mã quầy
 - **Hành vi riêng:** Mở tài khoản, nộp/rút tiền, in sao kê
- **Nhân viên tín dụng**
 - **Thuộc tính riêng:** Danh sách khách hàng phụ trách, hạn mức phê duyệt
 - **Hành vi riêng:** Tư vấn khoản vay, thẩm định hồ sơ, giải ngân
- **Nhân viên kiểm soát nội bộ**
 - **Thuộc tính riêng:** Phạm vi kiểm soát, báo cáo kiểm toán
 - **Hành vi riêng:** Kiểm tra chứng từ, phát hiện sai sót, đề xuất khắc phục

LỚP TRỪU TƯỢNG

- ❖ Là lớp mà nó không thể trở thành một lớp thực tế nào
- ❖ Được thiết kế nhằm tạo ra lớp có đặc tính tổng quát
- ❖ Bản thân nó chưa có ý nghĩa nên chưa thể viết mã cho đối tượng
- ❖ Ví dụ: bạn muốn tạo một hệ thống quản lý **phương tiện giao thông**
 - Xe nào cũng **có thể chạy**, nhưng cách chạy thì khác nhau (*xe đạp đạp bàn đạp, ô tô nổ máy, tàu thủy chạy bằng động cơ nước...*).
 - Chúng ta tạo lớp trừu tượng **Phuongtien** để quy định “Mọi phương tiện phải có hàm” **dichuyen()**, nhưng không nói cách di chuyển cụ thể - để lớp con tự định nghĩa

LỚP TRỪU TƯỢNG

```
#include <iostream>
using namespace std;

// Lớp trừu tượng
class PhuongTien {
public:
    // Phương thức thuần ảo (chưa có nội dung)
    virtual void diChuyen() = 0;

    // Phương thức bình thường (có nội dung)
    void baoGio() {
        cout << "Luôn tuân thủ luật giao thông!\n";
    }
};

// Lớp con: Xe đạp
class XeDap : public PhuongTien {
public:
    void diChuyen() override {
        cout << "Xe đạp di chuyển bằng cách đạp bàn đạp.\n";
    }
};

// Lớp con: Ô tô
class OTo : public PhuongTien {
public:
    void diChuyen() override {
        cout << "Ô tô di chuyển bằng cách nổ máy và đạp ga.\n";
    }
};

int main() {
    XeDap xd;
    OTo oto;

    xd.baoGio();
    xd.diChuyen();

    oto.baoGio();
    oto.diChuyen();

    return 0;
}
```

- ❖ Thông điệp:
 - Là phương tiện để đối tượng này chuyển yêu cầu tới đối tượng khác.
- ❖ Một thông điệp bao gồm:
 - Handle của đối tượng đích (đối tượng chủ)
 - Tên phương thức cần thực hiện
 - Các thông tin cần thiết khác (tham số)
- ❖ Hệ thống yêu cầu đối tượng thực hiện phương thức như sau:
 - Gửi thông báo và tham số cho đối tượng
 - Kiểm tra tính hợp lệ của thông báo
 - Gọi thực hiện hàm tương ứng phương thức

TRUYỀN THÔNG ĐIỆN

- ❖ Khách hàng muốn rút tiền từ tài khoản ATM.
- ❖ Thông điệp được gửi đi
 - **Handle của đối tượng đích:** Máy ATM (đang thao tác với tài khoản của khách hàng)
 - **Tên phương thức cần thực hiện:** rutTien()
 - **Các thông tin cần thiết (tham số):**
 - Số tiền cần rút: 2.000.000 VNĐ
 - Mã PIN: 20111983
- ❖ Hệ thống xử lý thông điệp
 - Máy ATM **nhận thông điệp** từ khách hàng (yêu cầu rút tiền).
 - **Kiểm tra tính hợp lệ** của thông điệp:
 - Mã PIN đúng không?
 - Số dư tài khoản có đủ không?
 - Nếu hợp lệ → **Gọi thực hiện phương thức:** ATM nhả tiền 2.000.000 VNĐ và in biên lai.

❖ Khái niệm:

- Là khả năng bỏ qua hay không để ý tới các thành phần không quan trọng.

❖ Các loại trừu tượng hoá:

- Trừu tượng hoá dữ liệu: không quan tâm các chi tiết không quan trọng bên trong
- Trừu tượng hoá chức năng: không quan tâm làm thế nào để thực hiện công việc

❖ Tóm lại:

- Chỉ quan tâm tới đặc điểm quan trọng
- Bỏ qua các chi tiết không liên quan

Sự trừu tượng hóa

- ❖ **Trừu tượng hóa dữ liệu:** Che giấu chi tiết về *cách dữ liệu được lưu trữ*, chỉ để lộ thông tin cần thiết cho người dùng.
 - **Ví dụ ngân hàng:** Khi bạn mở **app ngân hàng** và thấy số dư 50 triệu, bạn không biết (và cũng không cần biết) tiền của mình được lưu trên **server** dạng số nhị phân, trong **cơ sở dữ liệu SQL**, hay được nhân bản trên nhiều máy chủ dự phòng.
- ❖ **Trừu tượng hóa chức năng:** Che giấu *cách thức thực hiện* của một hành động, chỉ cung cấp cách gọi hành động đó.
 - Ví dụ: Khi bạn nhấn nút "**In**" trên máy tính, bạn chỉ quan tâm là tài liệu sẽ được in ra. Bạn không quan tâm máy in sẽ xử lý dữ liệu ra sao, chuyển đổi định dạng thế nào, hay giao tiếp với driver máy in như thế nào.

❖ Khái niệm:

- Là cơ chế ràng buộc dữ liệu và các thao tác trên dữ liệu thành thể thống nhất.

❖ Đóng gói gồm:

- Bao gói: người dùng giao tiếp với hệ thống qua giao diện
- Che dấu: ngăn chặn các thao tác không được phép từ bên ngoài

❖ Ưu điểm:

- Quản lý sự thay đổi
- Bảo vệ dữ liệu

❖ Trong ATM:

- Người dùng chỉ cần nhập thẻ, mã PIN, và số tiền muốn rút.
- Bên trong ATM có nhiều quy trình ẩn: kiểm tra số dư, xác thực mã PIN, kết nối ngân hàng, kiểm tra số tiền khả dụng.
- Những quy trình này **được đóng gói**, người dùng **không thể** và **không cần** can thiệp.

Sự đóng gói

```
#include <iostream>
#include <string>
using namespace std;

class ATM {
private:
    string maPIN = "1234"; // PIN thật, được đóng gói
    double soDu = 5000000; // Số dư tài khoản

    bool xacThucPIN(string pinNhap) {
        return pinNhap == maPIN;
    }

    bool kiemTraSoDu(double soTien) {
        return soTien <= soDu;
    }

    void truTien(double soTien) {
        soDu -= soTien;
    }

public:
    void rutTien(string pinNhap, double soTien) {
        if (!xacThucPIN(pinNhap)) {
            cout << " Sai ma PIN!" << endl;
            return;
        }
        if (!kiemTraSoDu(soTien)) {
            cout << " So du khong du!" << endl;
            return;
        }
        truTien(soTien);
        cout << " Rut thanh cong " << soTien << " VND" << endl;
    }

    void xemSoDu() {
        cout << " So du hien tai: " << soDu << " VND" << endl;
    }
};

int main() {
    ATM atm;
    atm.rutTien("1234", 2000000);
    atm.xemSoDu();
    atm.rutTien("9999", 1000000); // Sai PIN
}
```

❖ Khái niệm:

- Khả năng cho phép xây dựng lớp mới được thừa hưởng các thuộc tính của lớp đã có

❖ Đặc điểm:

- Lớp nhận được có thể bổ sung các thành phần
- Hoặc định nghĩa là các thuộc tính của lớp cha

❖ Các loại kế thừa:

- Đơn kế thừa
- Đa kế thừa

❖ Đơn kế thừa – Ứng dụng đặt xe

▪ Lớp cha: **Phuongtiengiaothong**

- Thuộc tính: biển số, tốc độ tối đa, hãng sản xuất
- Phương thức: khởi động(), dừng(), báo tình trạng()

▪ Lớp con: **Xeoto**

- Thừa hưởng tất cả từ **Phuongtiengiaothong**
- Bổ sung: số chỗ ngồi, mở điều hòa(), mở nhạc()

- ❖ Đa kế thừa – **Máy bán hàng tự động thông minh**
 - Lớp cha1: **Maytinhtien** – xử lý thanh toán, in hóa đơn.
 - Lớp cha 2: **Khohang** - quản lý tồn kho, cập nhật số lượng
 - **Lớp con: Maybanhangthongminh**
 - Vừa nhận tiền/in hóa đơn, vừa kiểm soát kho và cảnh báo khi gần hết hàng.

❖ Khái niệm:

- Khả năng đưa một phương thức có cùng tên trong các lớp con

❖ Thực hiện bởi:

- Định nghĩa lại
- Nạp chồng

❖ Cơ chế dựa trên sự kết gán:

- Kết gán sớm
- Kết gán muộn

- ❖ **Đa hình qua phương thức cùng tên – Hệ thống thanh toán**
 - Giả sử ta có phương thức thanhToan() nhưng cách xử lý khác nhau tùy loại hình thanh toán.
 - **Lớp cha: Thanhtoan**
 - **Phương thức: Thanhtoan()**
 - **Lớp con 1: Thanhtoantienmat** – in hóa đơn và thu tiền trực tiếp.
 - **Lớp con 2: Thanhtoanthenganghang** – quét thẻ, kiểm tra số dư, trừ tiền.
 - **Lớp con 3: Thanhtoanvidientu** – quét QR, gửi yêu cầu xác nhận OTP.

- ❖ **Đa hình qua kế thừa – Gửi thông báo**
 - **Lớp cha: Guithongbao** – có phương thức `send()`.
 - **Lớp con 1: Guiemail** – gửi qua SMTP.
 - **Lớp con 2: GuiSMS** – gửi qua API của nhà mạng.
 - **Lớp con 3: Guipushnotification** – gửi qua Firebase.
- ❖ **Phần mềm CRM (quản lý quan hệ khách hàng) thường chỉ cần gọi `send()` mà không cần biết đang gửi Email, SMS hay Push Notification, đây chính là đa hình.**